PhishGuard: An Explainable Hybrid System for Malicious Email Triage

Akshay Kumar Bolusani
ProjectKaro, Hyderabad, India
akshaykumarbolusani@gmail.com
https://github.com/AkshayKumarBolusani/PhishGuard-AI-Explainable-Hybrid-Phishing-Detection-System

***Abstract*—Security operations centers drown in suspicious email. Every false positive costs an analyst minutes they do not have. And a bare verdict such as "malicious: 92%" is useless in an incident report. There is nothing to quote and nothing to defend. PhishGuard is an open-source triage platform built around that reality. Its detection core pairs a deterministic 29-rule heuristic engine, aimed at phishing indicators, with TF-IDF statistical classifiers in a fixed weighted ensemble, and every decision ships with an analyst-readable report listing the evidence behind it.**

We benchmarked the core on five public corpora, 108,116 emails in total, spanning spam, phishing, and advance-fee fraud. The collection is curated for phishing-email research [6], [7], but the label semantics differ per corpus: the positive class means *spam* for TREC_07, CEAS_08, and SpamAssassin, *phishing* for Nazario_5, and *advance-fee fraud* for Nigerian_5. We report scores against each corpus's own positive class. No pooled "phishing detection" number is claimed. Evaluation uses stratified 5-fold cross-validation with identical splits for every model. Baselines: a PILFER-style heuristic feature set with random forest, TF-IDF with logistic regression, and TF-IDF with a linear SVM.

The hybrid reaches F1 scores of 0.908–0.986 on the per-corpus positive class and takes the best precision on four of five corpora (0.985–1.000). The TF-IDF SVM wins F1 on all five (0.984–0.998). That gap is the finding, not an embarrassment. The rule layer trades 1 to 14 points of recall for precision and per-indicator evidence. On these corpora and splits, where false alarms are expensive, that is a trade worth making. We do not claim it proves superiority in operational SOC environments. Code, protocol, and results are public.

***Index Terms*—malicious email detection, explainable AI, hybrid systems, SOC triage, spam and phishing**

I. Introduction

Phishing remains the most common first foothold in enterprise breaches. The detection side of the problem is well studied. The triage side is not. Once something looks suspicious, an analyst gets minutes to decide: escalate, contain, or dismiss. Two things decide whether a tool survives that moment. Precision comes first. A false positive is never free. It burns analyst time, breeds alert fatigue, and slowly teaches the team to ignore the tooling. Evidence comes second. A score with no reasoning cannot go into an incident report. The analyst needs to see why, in words they can quote.

The literature attacked detection from both ends. Fette *et al.*'s PILFER combined ten hand-crafted deception features with machine learning [1]. Zhang *et al.*'s CANTINA derived TF-IDF lexical signatures from page content and checked them against a search engine [2]. URL-lexical ML pipelines have held up as a durable baseline ever since [3]. Then explainability became a goal in its own right. EXPLICATE couples a classifier with LIME/SHAP explanations rendered into plain language by a large language model [4]. What we could not find was an open, deployable triage system aimed at the analyst instead of the end user, evaluated on public corpora spanning spam, phishing, and fraud, that reports the full cost of bolting a deterministic rule layer onto statistical classifiers rather than claiming a win on accuracy.

PhishGuard is that system. A FastAPI/React/MongoDB platform whose detection core fuses a 29-rule phishing-indicator engine with TF-IDF classifiers under a fixed weighted vote, with evidence-bearing analyst reports on every verdict. Three contributions. First, the system itself: deployable, precision-oriented email triage combining deterministic phishing indicators with statistical classifiers. Second, a public benchmark of the detection core, evaluated separately on public spam, phishing, and advance-fee-fraud corpora (108,116 emails), against three baselines, all on identical cross-validation splits, with per-corpus label semantics stated explicitly. Third, an honest account of where the hybrid helps (precision, evidence) and where it hurts (recall, F1) next to pure statistical models. Everything needed to reproduce the work is released.

Section II covers related work, Section III the architecture, Section IV the evaluation protocol, Section V the results, Section VI limitations and future work, and Section VII concludes.

II. Related Work

Heuristics and statistical detection. PILFER [1] introduced ten features aimed at deceptive technique: IP-based URLs, domain age, mismatched link targets, link and domain counts. With SVM and random-forest classifiers it reported over 96% phishing detection at roughly 0.1% false positives against the filters of its day. CANTINA [2] took the content route instead, deriving TF-IDF lexical signatures from page text and verifying them through a search engine. Sahingoz *et al.* [3] later showed that URL-lexical features with classical classifiers remain competitive. Our PILFER-style baseline reimplements that heuristic spirit with ten URL/HTML features. Domain-age and WHOIS lookups are left out; the evaluation ran with no network access.

Explainable phishing detection. EXPLICATE [4] pairs a classifier built on domain-specific features with a dual LIME/SHAP explanation layer, translated by DeepSeek v3 into accessible language. It reports 98.4% accuracy alongside measured explanation quality, shipped as a GUI and browser extension for end users. The audience is the difference. PhishGuard writes for the SOC analyst: indicators with quoted matched text, URL and sender findings, a recommended action. The explanation lives inside an alert-triage workflow, not a consumer tool. We describe these reports as a system feature; without a user study we make no claim that they measurably improve analyst decisions or save time.

Evaluation practice. Too many detectors are evaluated on private or single corpora. Comparison becomes impossible. The TREC Spam Track was an early attempt at a shared framework with public corpora such as trec07p [5]. We evaluate on the public *Phishing Email Curated Datasets* collection (Zenodo DOI 10.5281/zenodo.8339691) [6], [7], which spans the TREC 2007 and CEAS 2008 tracks plus Nazario, Nigerian-fraud, and SpamAssassin messages. The collection is presented for phishing-email research, but its constituent labels are not all phishing labels; Section IV states the per-corpus semantics explicitly, and the exact protocol and code are published with this paper.

III. System Architecture

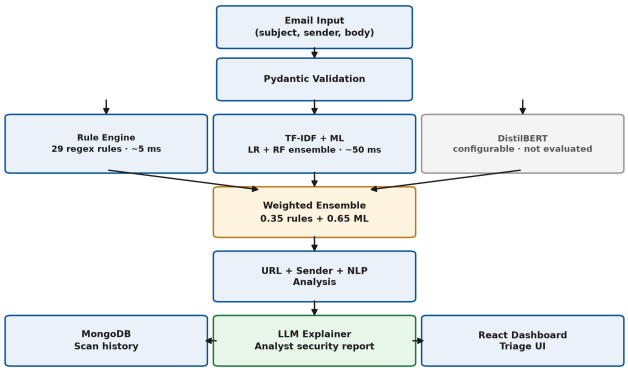


Fig. 1. PhishGuard pipeline. Email input passes validation, three detection layers (29-rule engine, TF-IDF ML ensemble, configurable DistilBERT), a fixed weighted ensemble, URL/sender/NLP analysis, and LLM explanation, served via REST API with a React triage dashboard and MongoDB history.

Fig. 1 shows the pipeline. Three design goals shaped it. No black boxes: whatever the system claims, the analyst must see why. Deterministic signals fire fast and never depend on model state. And the whole thing runs self-hosted, triaging each email in under a second.

A. Rule Engine

29 compiled regular-expression rules in ten categories (Table I), aimed at phishing indicators: credential harvesting, brand impersonation, urgency and threat language, financial-fraud lures, and technical deception. Each match carries a severity, low to critical, and a confidence. A 0–100 risk score and 3-class probabilities (safe/suspicious/phishing) come from severity-weighted aggregation. Typical latency is about 5 ms per email.

TABLE I
Rule-engine categories (29 rules total).

Category	Rules	Example pattern
Urgency	2	“act now”, “within 24 hours”
Threat	3	account suspension, legal threats
Credential harvesting	3	“verify your password”
Financial fraud	5	wire transfer, gift cards, crypto
Scam	4	lottery, tax refund, employment
Business-email compromise	2	CEO fraud, authority impersonation
Social engineering	3	emotional manipulation, curiosity bait
Link manipulation	2	“click here”, attachments
Technical deception	2	brand impersonation, HTML obfuscation
Style	3	excessive caps, punctuation, grammar

B. ML Classifier

TF-IDF over the email text, 10,000 features, 1–2 grams, sublinear TF. On top of it sit logistic regression (C=1.0, class-weighted) and a random forest (100 trees, max depth 12); their probabilities are averaged. Per-word coefficients are exposed so the dashboard can show feature importance. Typical latency is about 50 ms.

C. Ensemble

Without the optional transformer, the final score is a fixed weighted vote:

$$P = 0.35 \cdot P_{\text{rules}} + 0.65 \cdot P_{\text{ML}}, \tag{1}$$

with P_{ML} the averaged LR/RF probability. A DistilBERT layer exists at weight 0.40, rebalancing the mix to 0.25/0.35/0.40, but it was not trained for this build. It is switched off and excluded from every number reported here. The weights are shipped defaults. They were not tuned on the benchmark corpora.

D. Explanation and Platform

URL analysis scores every link for suspiciousness. Sender-trust scoring and NLP urgency features complete the picture. An LLM explainer turns all of it into an analyst-style report: executive summary, classification with confidence, indicators with quoted evidence, URL and sender findings, attack type, recommended actions. These reports are a system feature; we do not claim measured improvements in analyst speed or decision quality. Around the core sits the platform. 17 REST endpoints. A React dashboard with threat overview, split-pane scanner, and searchable history exporting to CSV and JSON. MongoDB persistence. A feedback endpoint where analysts can correct verdicts.

IV. Evaluation

A. Corpora and label semantics

Table II summarizes the five corpora from [6], [7] (1 = positive class, 0 = negative class). The collection is curated for phishing-email research, but the positive class does *not* mean phishing everywhere. For TREC_07, CEAS_08, and SpamAssassin it means *spam*, inherited from their source spam evaluations (TREC 2007 Spam Track public corpus trec07p [5], the CEAS 2008 challenge, and the SpamAssassin public corpus). For Nazario_5 it means *phishing*, and for Nigerian_5 *advance-fee fraud*. We verified the semantics against the source tracks and by inspecting labeled examples (e.g., TREC_07 positives are pharmaceutical spam; Nazario_5 positives are credential-harvesting lures). Every file was MD5-checked against the Zenodo-published checksums (v1). Email text is subject plus body. HTML is kept as-is so the link-manipulation features see the original markup.

TABLE II
PUBLIC EVALUATION CORPORA. THE POSITIVE CLASS IS CORPUS-SPECIFIC; SCORES ARE REPORTED PER CORPUS, NEVER POOLED INTO A SINGLE CLAIM.

Corpus	Messages	Pos.	Legit.	Positive =
TREC_07	53,757	29,399	24,358	spam
CEAS_08	39,154	21,842	17,312	spam
Nazario_5	3,065	1,565	1,500	phishing
Nigerian_5	6,331	3,332	2,999	advance-fee fraud
SpamAssassin	5,809	1,718	4,091	spam
Total	108,116	57,856	50,260	—

B. Protocol

Stratified 5-fold cross-validation, random state 42, with the **same splits for every model**. The TF-IDF vectorizer is fit on each training fold only. No test vocabulary leaks into training. We report precision, recall, and F1 on the positive class in Table III, all on pooled out-of-fold predictions; the released benchmark outputs additionally record accuracy and macro F1 for every model. The corpora are binary, so the system's *suspicious* class is not exercised here. On leakage: the heuristic rule engine is a deterministic, non-fitted component built from predefined regular-expression rules with hardcoded severity weights. Its scores are computed independently for each email and use no labels, corpus-level statistics, or information from the evaluation folds. TF-IDF vectorization and classifier fitting use only the training partition of each fold, and the resulting transformations are applied to that fold's held-out partition. Exact duplicates are rare (0.14% of TREC_07 at most, none with conflicting labels); a deduplicated-before-split sensitivity rerun is reported in Section V.

C. Baselines

PILFER-style: ten hand-crafted URL/HTML features. IP-literal URL, URL and domain counts, mismatched anchor/href, '@' in URL, subdomain depth, HTML forms, JavaScript, longest URL, HTTPS ratio. Balanced random forest. It reimplements the [1] idea, not the verbatim feature set. *TF-IDF+LR:* the system's own ML configuration, retrained per fold. *TF-IDF+SVM:* the same TF-IDF features with a balanced linear SVM. *PhishGuard hybrid:* the production rule engine, imported verbatim from the codebase, combined per (1) and thresholded at 0.5.

The repository also ships a 130-email demonstration set, 50 safe, 35 suspicious, 45 phishing, for pipeline validation. Its cross-validation figures (LR F1 0.83 ± 0.06, RF F1 0.85 ± 0.09) describe the demo. They are **not** benchmark accuracy and are not presented as results.

V. Results

Table III reports precision, recall, and F1 on the positive class per corpus. Fig. 2 plots the F1 comparison. All scores are empirical results on these corpora and splits.

TABLE III
BENCHMARK RESULTS: PRECISION / RECALL / F1 ON THE POSITIVE CLASS (SPAM, PHISHING, OR ADVANCE-FEE FRAUD DEPENDING ON CORPUS; POOLED 5-FOLD OUT-OF-FOLD PREDICTIONS). BOLD MARKS THE BEST F1 AND THE BEST PRECISION WITHIN EACH CORPUS BLOCK.

Model	Precision	Recall	F1
TREC_07 — 53,757 emails (positive = spam)			
PILFER-style RF	0.791	0.964	0.869
TF-IDF + LR	0.991	0.997	0.994
TF-IDF + SVM	0.995	0.998	**0.996**
PhishGuard hybrid	**0.996**	0.976	0.986
CEAS_08 — 39,154 emails (positive = spam)			
PILFER-style RF	0.949	0.665	0.782
TF-IDF + LR	0.996	0.995	0.996
TF-IDF + SVM	0.998	0.998	**0.998**
PhishGuard hybrid	**0.999**	0.962	0.980
Nazario_5 — 3,065 emails (positive = phishing)			
PILFER-style RF	0.656	0.957	0.778
TF-IDF + LR	0.993	0.989	0.991
TF-IDF + SVM	0.997	0.994	**0.996**
PhishGuard hybrid	**0.998**	0.877	0.933
Nigerian_5 — 6,331 emails (positive = fraud)			
PILFER-style RF	0.624	0.908	0.740
TF-IDF + LR	1.000	0.986	0.993
TF-IDF + SVM	1.000	0.991	**0.996**
PhishGuard hybrid	1.000	0.960	0.980
SpamAssassin — 5,809 emails (positive = spam)			
PILFER-style RF	0.647	0.836	0.730
TF-IDF + LR	0.966	0.980	0.973
TF-IDF + SVM	0.982	0.986	**0.984**
PhishGuard hybrid	**0.985**	0.842	0.908

The two large spam corpora tell a consistent story. On TREC_07 and CEAS_08 every model is strong. The hybrid takes the best precision observed anywhere, 0.996 and 0.999, with F1 of 0.986 and 0.980. Just behind the TF-IDF SVM on both. On the phishing corpus, Nazario_5, the cost appears: hybrid recall drops to 0.877 while precision holds at 0.998. The rule layer's conservatism vetoes true positives its statistical half would have caught. SpamAssassin is the hardest case for the hybrid: recall 0.842, F1 0.908. That makes sense. It is general spam, not phishing, and the rule engine is tuned for

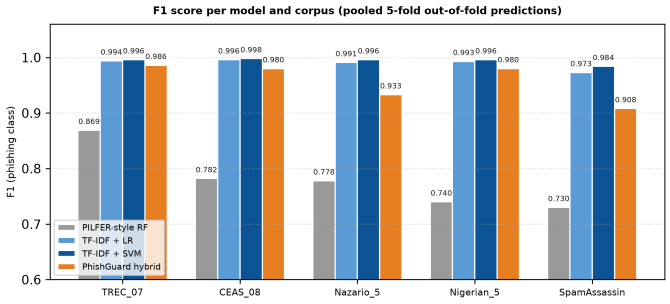


Fig. 2. F1 score per model and corpus, on each corpus's positive class (pooled 5-fold out-of-fold predictions).

phishing indicators, so its contribution shrinks. The PILFER-style baseline behaves the way classical heuristics do. High recall, 0.836 to 0.964, with clearly lower precision, 0.624 to 0.949.

No claim of accuracy superiority here. The hybrid does not win on F1. What the numbers show is a documented trade, measured separately on spam, phishing, and fraud corpora. The deterministic rule layer buys the highest precision and per-indicator evidence at a recall cost of 1 to 14 points depending on the corpus. On these corpora and splits, for triage work where false positives are expensive analyst time, that is a defensible place to stand. It is not proof of superiority in an operational SOC.

A. Duplicate sensitivity

Exact duplicates (normalized subject plus body) are rare: 74 of 53,757 messages in TREC_07 at most, none with conflicting labels. To check whether duplicates crossing train/test folds inflate the scores, we reran the full protocol with exact duplicates removed *before* the stratified split (53,683 kept for TREC_07; 39,145 for CEAS_08; 3,063 for Nazario_5; 6,315 for Nigerian_5; 5,809 for SpamAssassin), keeping every other setting identical. The largest per-model F1 movement was 0.0029 (Nazario_5 hybrid, 0.9333 to 0.9362); all other deltas were at most 0.0012, and SpamAssassin was bit-identical. The best-precision and best-F1 orderings were unchanged on all five corpora. Duplicates do not drive the findings.

VI. Discussion, Limitations, and Future Work

The corpora are binary. So the system's three-way output, safe/suspicious/phishing, could not be fully evaluated. How the *suspicious* band behaves in practice is unmeasured. A graded-alert corpus would be a worthwhile follow-up. The DistilBERT layer sat out this build, untrained. Running it through the same protocol is the obvious next experiment. The rule-engine thresholds and the 0.35/0.65 ensemble weights are shipped defaults, never tuned. Per-deployment calibration should recover some of the recall gap. Cross-dataset evaluation, training on one corpus and testing on another, would test generalization harder than random folds do; it is future work. Triage-time reduction in a real SOC is unmeasured. A protocol for the author's SOC lab exists: 30 real Wazuh alerts, baseline versus assisted triage, timed, with a washout period. The study has not run, so no such claim is made. Finally, the corpora are English and partly dated. Modern spear-phishing, non-English lures, and adversarial evasion of regex rules are all untested.

VII. Conclusion

PhishGuard is an open, deployable malicious-email triage platform. Its hybrid core, evaluated separately on public spam, phishing, and advance-fee-fraud corpora (108,116 emails) against three baselines, leads on precision at 0.985 to 1.000 while trailing pure TF-IDF models on F1. An honest, measured trade-off on these corpora and splits. The system ships with evidence-bearing analyst reports as a built-in feature. Code, protocol, and results are released for reproduction.

Acknowledgment

The manuscript prose was drafted with the assistance of a generative AI system (Muse, Meta) working from the author's system design, source code, and measured benchmark results; the figures were produced with AI-assisted scripting. All experimental results, numbers, and technical claims derive from the author's own work. AI assistance was limited to drafting and language; the research contribution is the author's own.

References

[1] I. Fette, N. Sadeh, and A. Tomasic, "Learning to detect phishing emails," in *Proc. 16th Int. Conf. World Wide Web (WWW)*, Banff, AB, Canada, 2007, pp. 649–656. doi: 10.1145/1242572.1242660.

[2] Y. Zhang, J. Hong, and L. Cranor, "Cantina: a content-based approach to detecting phishing web sites," in *Proc. 16th Int. Conf. World Wide Web (WWW)*, Banff, AB, Canada, 2007.

[3] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, "Machine learning based phishing detection from URLs," *Expert Syst. Appl.*, vol. 117, pp. 345–357, Mar. 2019. doi: 10.1016/j.eswa.2018.09.029.

[4] B. Lim, R. Huerta, A. Sotelo, A. Quintela, and P. Kumar, "EXPLICATE: Enhancing phishing detection through explainable AI and LLM-powered interpretability," *arXiv preprint arXiv:2503.20796 [cs.CR]*, 2025.

[5] G. V. Cormack, "TREC 2007 Spam Track overview," in *Proc. 16th Text REtrieval Conf. (TREC)*, NIST, Gaithersburg, MD, 2007.

[6] *Phishing Email Curated Datasets*, Zenodo, doi: 10.5281/zenodo.8339691, 2023. CC-BY-4.0.

[7] A. I. Champa, M. F. Rabbi, and M. F. Zibran, "Curated datasets and feature analysis for phishing email detection with machine learning," in *Proc. 3rd IEEE Int. Conf. Comput. and Mach. Intell. (ICMI)*, 2024, pp. 1–7.